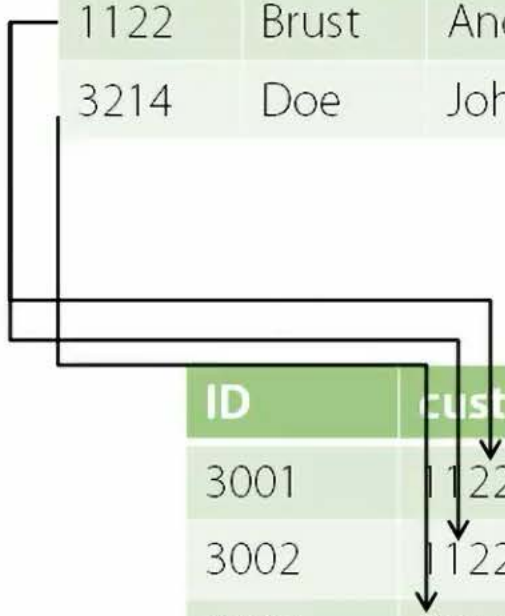
Relational (Conventional) Databases

Customers

ID	lname	fname	address	city	state	zip
1122	Brust	Andrew	123 Main St.	New York	NY	10099
3214	Doe	John	321 Elm St.	Anytown	MI	40001

Orders

ID	custid	amount	tax	shipdate
3001	1122	500	40	2/20/2012
3002	1122	250	17	3/18/2012
3003	3214	1700	150	3/20/2012



What is NoSQL?

Customers

ID: 1122	**lname:** Brust	**fname:** Andrew	**address:** 123 Main St.	**city:** New York	**state:** NY	**zip:** 10099
ID: 3214	**lname:** Doe	**fname:** John	**address:** 321 Waterford Crescent.	**village:** Stodday	**county:** Lancashire	**postal code:** LA2 6ET

Orders

ID: 3001	**customerID:** 1122	**amount:** 500	**tax:** 40	**processdate:** 2/20/2012
ID: 3002	**customerID:** 1122	**amount:** 250	**shipdate:** 3/18/2012	
ID: 3003	**amount:** 1700	**tax:** 150		

NoSQL Data Fodder

"Web Scale"

- This the term used to justify NoSQL
- Scenario is simple needs but "made up for in volume"
  - Millions of *concurrent* users
- Think of sites like Amazon or Google
- Think of non-transactional tasks like loading catalog data to display product page, or environment preferences

What is NoSQL?

Customers

ID: 1122	**lname:** Brust	**fname:** Andrew	**address:** 123 Main St.	**city:** New York	**state:** NY	**zip:** 10099
ID: 3214	**lname:** Doe	**fname:** John	**address:** 321 Waterford Crescent.	**village:** Stodday	**county:** Lancashire	**postal code:** LA2 6ET

Orders

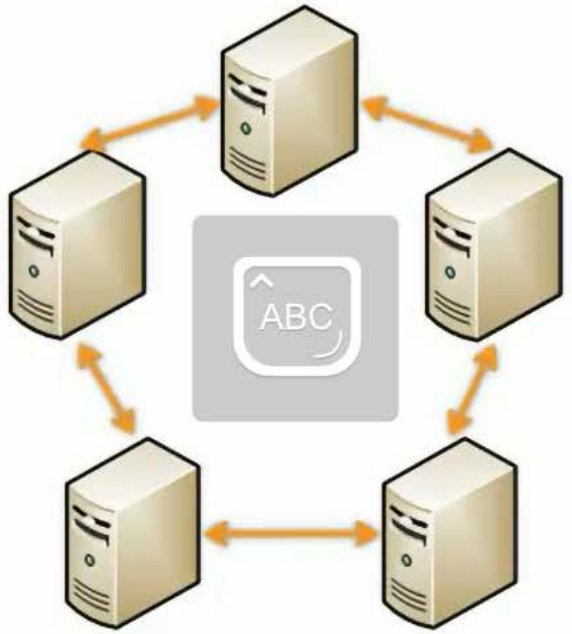
ID: 3001	**customerID:** 1122	**amount:** 500	**tax:** 40	**processdate:** 2/20/2012
ID: 3002	**customerID:** 1122	**amount:** 250	**shipdate:** 3/18/2012	
ID: 3003	**amount:** 1700	**tax:** 150		

Distributed Architecture

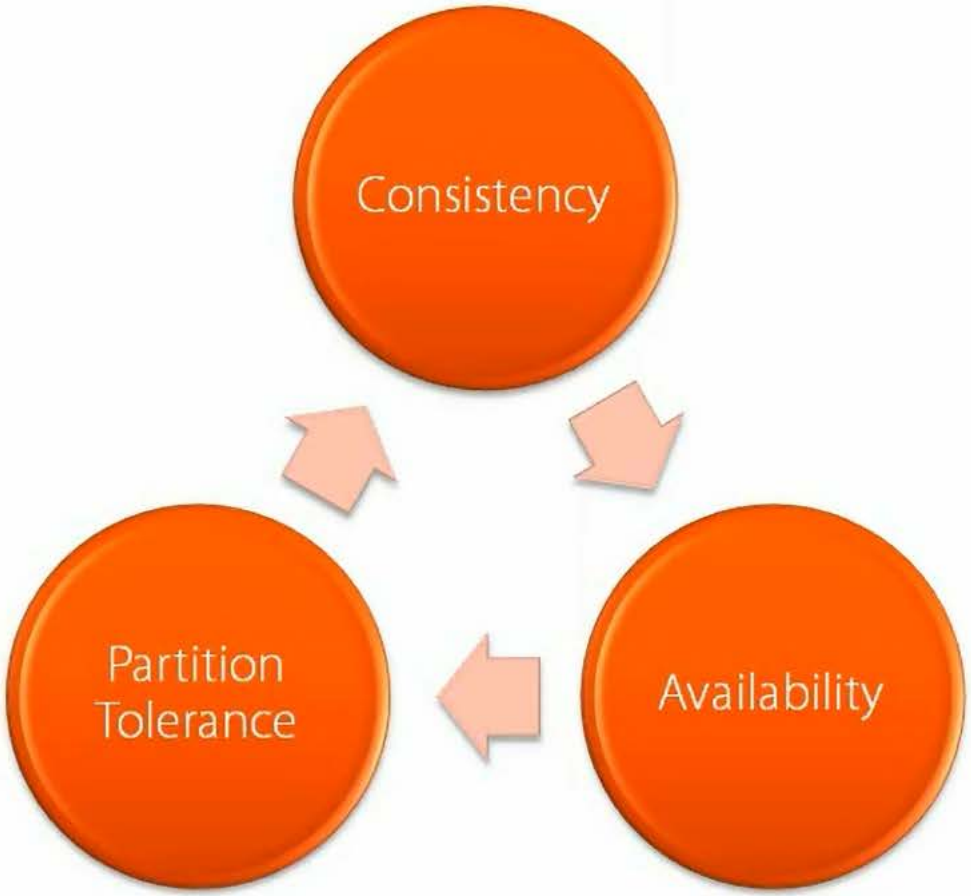
- Many NoSQL Databases federate a number of commodity server together
- Provides redundant storage
- Provides geographic distribution
- Avoids having a "single point of failure."

Distributed Architecture

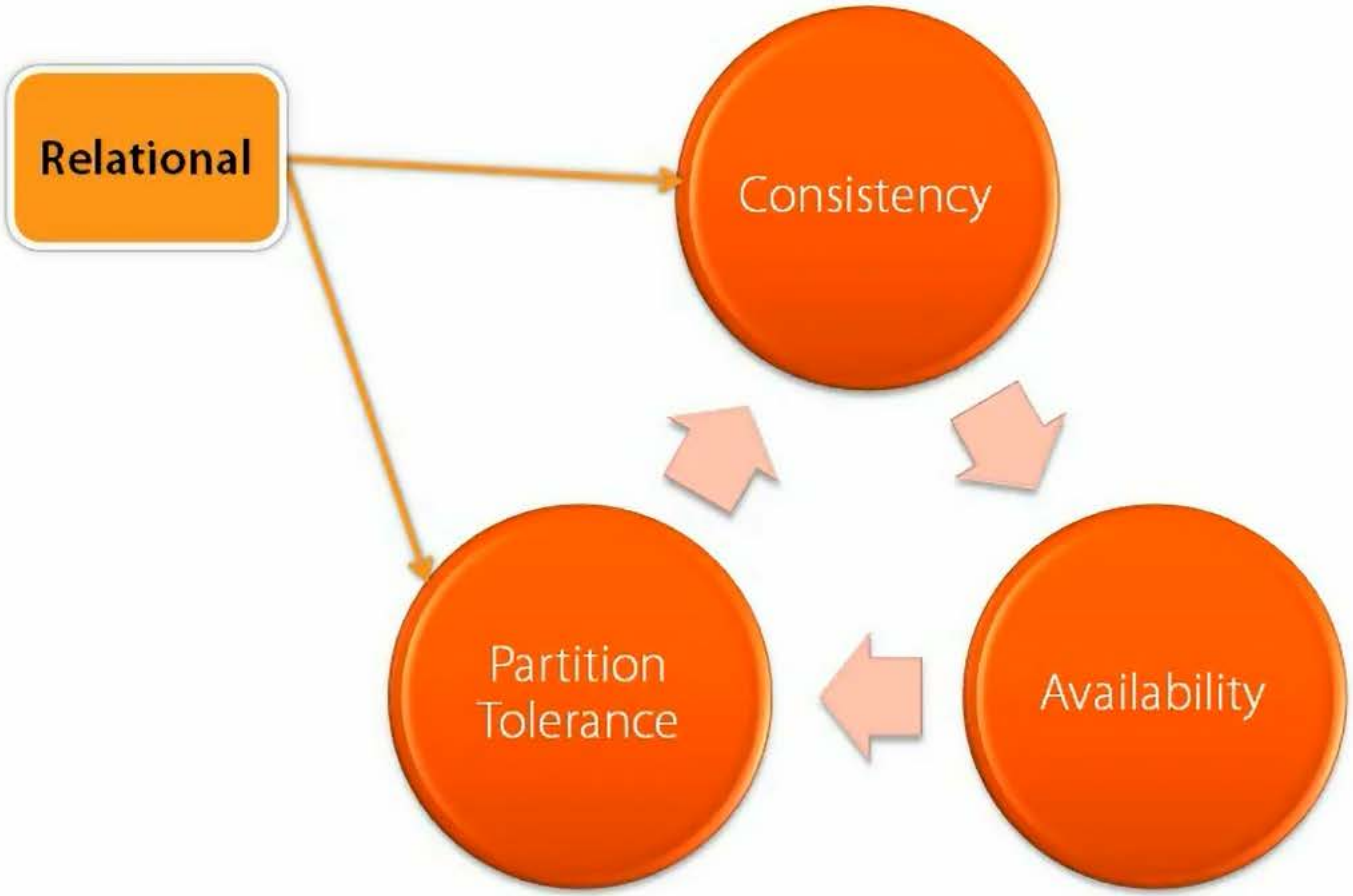
- Many NoSQL Databases federate a number of commodity server together
- Provides redundant storage
- Provides geographic distribution
- Avoids having a "single point of failure."



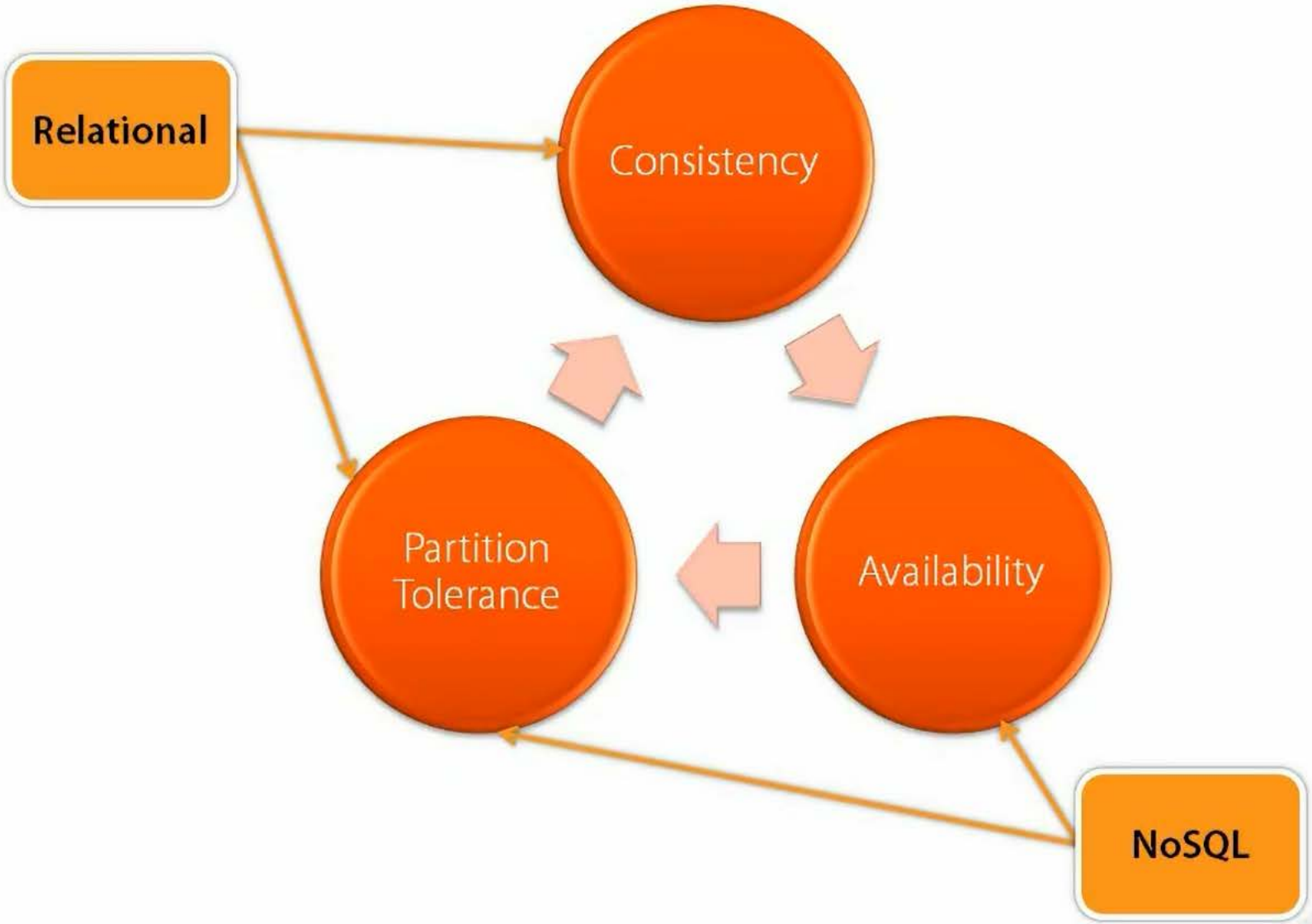
CAP Theorem



CAP Theorem



CAP Theorem



Consistency

- **Things like inventory, account balances should be consistent**
  - Imagine updating a server in Seattle that stock was depleted
  - Imagine *not* updating the server in NY
  - Customer in NY goes to order 50 pieces of the item
  - Order processed even though no stock
- **Things like catalog information don't have to be, at least not immediately**
  - If a new item is entered into t he catalog, it's OK for some customers to see it even before the other customers' server know about it
- **But catalog info must come up quickly**
  - Therefore don't lock data in one location while waiting to update he other
- **Therefore, OK to sacrifice consistency for speed, in some cases**

Compromises

- Eventual consistency
- Write buffering
- Only primary keys can be indexed
- Queries must be written as programs
- Tooling
  - Productivity (= money)

NoSQL Queries

- Typically no query language
- Instead, create procedural program
- Sometimes SQL is supported
- But usually MapReduce code is used…

Cloud

- **Microsoft**
  - Azure Tables
  - Hadoop on Azure/Hbase
- **Amazon**
  - SimpleDB
  - DynamoDB
  - Elastic MapReduce
- **DB-specific NoSQL Hosters**
  - E.g. MongoHQ for MongoDB
  - CouchDB on Cloudant
- **DIY on Infrastruture as a Service (IaaS)**

NoSQL Categories

- **Key-Value Stores**
  - e.g. Name: "Andrew"
  - Schema-free (every record can have different keys)
  - Most common, basis for other three
- **Document Stores**
  - Everything's in a document, including other documents
  - Schema-free
- **Wide Column Stores**
  - Know the groups of columns, but not columns themselves
  - Semi-schematic
- **Graph Databases**
  - Relationship-focused